

PARCIAL 1 SISTEMAS OPERATIVOS

MIGUEL ANGEL AGUILAR

18 DE AGOSTO

EAFIT

2025

Se desarrollaron dos programas, uno en C++ y otro en C, con el propósito de generar información de personas incluyendo atributos como nombre, apellido, número de

identificación, ciudad y fecha de nacimiento, además de variables económicas como ingresos anuales, patrimonio, deudas y la condición de si presentan o no declaración de renta. Para la generación de estos datos se implementaron funciones específicas que permiten crear fechas aleatorias, asignar identificaciones únicas y producir números aleatorios para los valores financieros, garantizando así diversidad en los registros generados.

Las pruebas se realizaron en el sistema operativo Kali Linux, compilando ambos programas en sus respectivos lenguajes. Cada implementación se ejecutó en dos modalidades distintas: mediante punteros, donde se trabaja con referencias para evitar la copia de datos, y mediante paso por valores, donde se copian todos los datos completos. El análisis de rendimiento incluyó la medición del tiempo de ejecución, utilizando chrono en C++ y timeval en C, así como el consumo de memoria, evaluado con la herramienta rusage.

PREGUNTAS OBLIGATORIAS

1. Persona mas longeva

En esta parte lo probe con 1 millón de registros, primero lo probé en C++ y la persona mas longeva fue Daniela Valencia López que nació en 1961 así que tiene 64 años, ya luego lo probe en C y salió Alejandro Sánchez Martínez de 1960 así que tiene 65 años.

2. Persona con mas patrimonio

Volvemos a buscar con 1 millón de registros y vemos que la persona con mas patrimonio en C++ fue Pedro González Castro con \$1,998,057,944.63 y en C fue David Ramírez Vargas con \$9,998,057,944.63

3. Declarantes de renta y promedios

La cantidad de declarantes de renta, así como los promedios de patrimonio e ingresos por grupo, presentan ligeras variaciones entre cada ejecución debido al carácter aleatorio de los datos, aunque los resultados obtenidos en C++ y en C son muy similares. Por

ejemplo, al generar un millón de registros, el número de declarantes se ubica aproximadamente en 200,000 para los grupos A y B, y cerca de 137,000 para el grupo C. En cuanto a los ingresos, los promedios rondan los \$255,000,000 por grupo.

PREGUNTAS ADICIONALES

1. Ciudades con mayor patrimonio promedio

Para un millón de registros estos fueron los resultados Medellín (\$878,038,238.82), Neiva (\$823,265,238.34), Bogotá (\$732,475,234.56).

Los patrimonios promedio por ciudad muestran valores relativamente cercanos entre Medellín (\$878M) y Neiva (\$823M), lo que sugiere una distribución balanceada en esos dos casos. Sin embargo, Bogotá presenta un promedio más bajo (\$732M), lo que indica que en esta ciudad los patrimonios generados tienden a concentrarse en valores menores en comparación con las otras dos. Esta diferencia podría deberse al carácter aleatorio en la asignación de patrimonios, que no privilegia ninguna ciudad en particular y, por lo tanto, genera variaciones naturales en los promedios. Aun así, la magnitud de los resultados es consistente en el orden de cientos de millones, lo que refuerza la idea de que el método escala adecuadamente cuando se trabaja con un millón de registros.

2. Porcentajes de personas mayores a 60 años.

Con la generación de 1 millón de registros 1960 y 2010, las edades oscilan entre 15 y 65 años. Esto implica que solo quienes nacieron entre 1960 y 1965 superan los 60 años, lo que representa alrededor del 11.76% del total. Así, en un conjunto de un millón de registros se espera en promedio cerca de 117,600 personas mayores de 60 años. Este resultado muestra que, aunque el rango de edades es amplio, la proporción de adultos mayores dentro de los datos es relativamente baja, lo cual puede influir en análisis demográficos o proyecciones sociales basadas en la distribución generada.

3. Persona mas joven de Colombia

Para un millón de registros encontramos que la persona mas joven de Colombia fue Laura Rodríguez García que nació en 2010 que es la fecha limite que tenemos así que tiene 15 años.

PREGUNTAS DE PENSAMIENTO CRITICO

Memoria

El uso de apuntadores reduce hasta un 75% de la memoria porque en lugar de copiar estructuras completas de datos, se almacenan y manipulan solo referencias (direcciones de memoria). Esto significa que no se duplica la información en cada operación y se evita que cada variable contenga una copia de todos los campos. En grandes volúmenes (como 10 millones de registros) esta diferencia se amplifica, ya que trabajar con valores implica reservar espacio completo para cada objeto, mientras que con punteros solo se guarda la dirección de referencia, optimizando de forma drástica el consumo de memoria.

Datos

Si los grupos dependen del ID, lo más eficiente es separar los registros desde la inserción en listas A, B y C revisando solo los últimos dígitos. Así, las búsquedas se hacen directamente en la lista correspondiente y no en todo el conjunto, acelerando consultas en grandes volúmenes.

Localidad

El acceso a memoria difiere entre un array de structs y un vector de clases por cómo se organizan los datos en RAM. En un array de structs, los campos de cada registro están contiguos, lo que favorece la localidad espacial: el procesador aprovecha las cachés al acceder secuencialmente. En cambio, con un vector de clases (particularmente si internamente usan punteros dinámicos), los datos pueden estar dispersos en memoria, reduciendo la eficiencia de la caché y aumentando los fallos de acceso. Esto explica por qué en operaciones masivas el array de structs suele ser más rápido y escalable.

Escalabilidad

Cuando los datos exceden la memoria RAM disponible, se puede recurrir a mmap o a la memoria virtual para mapear archivos completos a la dirección de memoria del proceso. Esto permite acceder a los datos como si fueran parte de la RAM, pero en realidad se cargan bajo demanda desde disco. Así, no es necesario cargar todo el dataset en memoria, sino solo las páginas activamente utilizadas. Esto hace posible procesar volúmenes mucho mayores que la RAM física, aunque con la penalización de latencia al acceder a disco, por lo que resulta clave diseñar accesos secuenciales o en bloques grandes para minimizar los costos de swapping.

CONCLUSIONES

El programa en C resulta más rápido y también más eficiente en el uso de memoria, sobre todo cuando se implementan apuntadores, ya que se evita la duplicación de datos y se aprovecha mejor la localidad de memoria. En cambio, C++, aunque ofrece mayor facilidad de uso gracias a sus librerías y a un manejo más intuitivo de estructuras complejas, presenta un desempeño más lento y un consumo de memoria mayor. Esto refleja la clásica diferencia entre ambos lenguajes: C está más cerca del hardware y prioriza la eficiencia, mientras que C++ sacrifica parte de esa velocidad para ofrecer abstracciones que simplifican el desarrollo y mejoran la legibilidad del código. Los datos que podemos generar no son tan alejados y son muy consistentes y esta es una muy buena manera de manejar grandes cantidades de datos.